This manual is designed for you (the owner) or your buyers to deploy Class Manager - IMS from scratch. It follows the logic of a Production Engineer: step-by-step, zero-waste, and fail-safe.

📘 Class Manager - IMS: Deployment Manual

Version: 1.0 (Enterprise Multi-Tenant Edition)
Infrastructure: GitHub (Web Hosting) + Supabase (Database & Auth)

🏗 Pre-deployment Checklist

Before you start, ensure you have:

A GitHub Account (Free) – This stores your "App code."

A Supabase Account (Free/Paid) – This is your "Digital Warehouse" (Database).

Cloudflare account (Free)

The index.html file (Rename your imp.html to index.html for easier hosting).

The manifest.json and sw.js files (for the PWA features).

ClassManager-IMS: Deployment & Setup Guide
Phase 1: Database Infrastructure (Supabase)

The database stores all student, teacher, and financial records.

Create Account: Go to Supabase.com and sign up for a Free Tier account.

Create Project: Create a new project named ClassManager-IMS. Set a strong database password and keep it safe.

Run Master Script:

In the Supabase Dashboard, click on SQL Editor (left menu).

Click "+ New Query".

Paste the Full Master Deployment Script provided in the previous step.

Click Run. All 24 tables and security policies are now created.

Configure Storage (For ID Cards & Documents):

Click on Storage (bucket icon) in the left menu.

Create a New Bucket named: scholarship_docs.

Set the bucket to "Public" (so students can view their uploaded documents).

Just like tables, buckets need security. In the Supabase Dashboard under Storage > Policies, tell the client to:

Click "New Policy" for the bucket.

Choose "Allow individual access" (so User A cannot see User B's uploaded PDFs).

```
-- 1. ALLOW PUBLIC TO VIEW (SO LINKS WORK IN EMAILS/CARDS)
CREATE POLICY "Public can view student docs" ON storage.objects
FOR SELECT TO public
USING (bucket_id = 'scholarship_docs');

-- 2. ALLOW STAFF/OWNERS TO UPLOAD
CREATE POLICY "Staff can upload student docs" ON storage.objects
FOR INSERT TO authenticated
WITH CHECK (bucket_id = 'scholarship_docs');

-- 3. ALLOW STAFF/OWNERS TO DELETE
CREATE POLICY "Staff can delete student docs" ON storage.objects
FOR DELETE TO authenticated
USING (bucket_id = 'scholarship_docs');
```

Gather API Credentials:

Go to Project Settings (gear icon) -> API.

Copy the Project URL and the anon public key. You will need these for the next phase.

Phase 2: API & Security (Cloudflare Worker)

The Cloudflare Worker acts as a secure bridge between your website and the database.

Create Account: Sign up at Cloudflare.com.

Create Worker:

Go to Workers & Pages -> Create Application -> Create Worker.

Name it classmanager-api. Click Deploy.

Click Edit Code and paste your Worker JavaScript code (the proxy code).

Set Environment Variables:

In the Worker dashboard, go to Settings -> Variables.

Add SUPABASE_URL (paste the URL from Phase 1).

Add SUPABASE_KEY (paste the Key from Phase 1).

Capture Worker URL: Your API will look like: https://classmanager-api.[yourname].workers.dev.

Phase 3: Source Code Configuration

Now you connect the HTML files to the new backend.

Update Files: Open your source code files (imp.html, time_table.html, etc.) in a text editor (like VS Code or Notepad).

Replace Credentials: Find the Supabase Initialization section at the top of the script and update it:

```
const _supabaseUrl = 'PASTE_YOUR_CLOUDFLARE_WORKER_URL_HERE';
const _supabaseKey = 'PASTE_YOUR_SUPABASE_ANON_KEY_HERE';
```

GitHub Hosting:

Create a new private or public repository on GitHub.

Upload all .html files and assets (images/logos).

Go to Settings -> Pages.

Set "Branch" to main and click Save.

This guide is for a Direct Connection between GoDaddy and GitHub (without using Cloudflare for DNS). This is the simplest way to get your professional domain pointing to your source code.

Step 1: Create & Configure GitHub Repository

1. Create a Repository
Log in to GitHub.com.
Click New (green button) to create a new repository.
Name it classmanager.
Set it to Public (required for free hosting).
Click Create repository.

2. Upload Files
Click the link "uploading an existing file".
Drag and drop your index.html, manifest.json, sw.js, and your logo image.
Click Commit changes.

Before touching GoDaddy, tell GitHub which domain it should expect.

Log in to GitHub and open your repository (classmanager-ims).

Click Settings (top tab) -> Pages (left menu).

Under Custom domain, type your domain (e.g., www.classmanager.tech).

Click Save.

GitHub will show a message: "DNS check in progress." (It will fail initially, which is normal).

Step 2: Configure GoDaddy DNS

Now you must point the "Address" at GoDaddy to the "House" at GitHub.

Log in to your GoDaddy Control Panel.

Go to Domain Manager and click on your domain name.

Click the DNS tab (or Manage DNS).

Delete any existing A or CNAME records named @ or www (especially those pointing to "Parked" or "Website Builder").

Add the 4 GitHub IP Addresses (A Records):
Click Add New Record for each of these. You need all four for high availability.

Type: A | Name: @ | Value: 185.199.108.153 | TTL: Default

Type: A | Name: @ | Value: 185.199.109.153 | TTL: Default

Type: A | Name: @ | Value: 185.199.110.153 | TTL: Default

Type: A | Name: @ | Value: 185.199.111.153 | TTL: Default

Add the WWW Alias (CNAME Record):
This ensures that when someone types www., they reach your site.

Type: CNAME | Name: www | Value: [your-github-username].github.io | TTL: Default
(Example: sunil8879.github.io)

Step 3: Verify and Secure (The Final Step)

Go back to your GitHub Settings -> Pages page.

Refresh the page. You should now see a green message: "DNS check successful" or "Domain is verified."

Check the Box: "Enforce HTTPS".

Note: If this box is greyed out, wait 15–30 minutes. GitHub is currently generating a security certificate for you. Once it becomes clickable, check it immediately.

Summary of DNS Table in GoDaddy

After your setup, your GoDaddy DNS table should look exactly like this:

Type	Name	Value	Purpose
A	@	185.199.108.153	Points root domain to GitHub
A	@	185.199.109.153	Points root domain to GitHub
A	@	185.199.110.153	Points root domain to GitHub
A	@	185.199.111.153	Points root domain to GitHub
CNAME	www	[username].github.io	Redirects www traffic
CNAME	_domainconnect	(Leave this one alone)	GoDaddy internal use

Troubleshooting

Propagation Time: DNS changes can take anywhere from 10 minutes to 2 hours to work globally. If you see a 404 error, just wait.

Relative Paths: Ensure all links in your HTML are relative (e.g., href="time_table.html" instead of href="/classmanager-ims/time_table.html").

Browser Cache: If you don't see changes, try opening your site in an Incognito/Private window.

Final Launch Checklist

Visit the domain. Does it show the Home page with a Padlock (HTTPS)?

Click Signup -> Choose Owner. Does it generate a new Institute ID?

Try to Add a Student. Does the record appear in the dashboard?

Click Fetch in the Time Table. Does it pull data from the database?

Setup Complete! Your Evergreen Institute Management System is now live.

💰 How to sell this to a new Institute (One-Time Sale)

If you sell this to a new institute and they want their own Supabase account:

Ask them to create a Supabase account (or do it for them).

Run Phase 1 (SQL Script) on their new account.

Change the 2 Keys in the index.html to their new keys.

Deploy a new version to a new GitHub link (or a custom domain).

Collect your payment.

Why this works for a Layman:

No Servers: You don't need to pay for hosting (GitHub is free).

No Maintenance: Since there is no backend code (Node.js/Python), nothing will "crash" or need a restart.

Direct Control: The institute owns its database. If you disappear, their data is still safe in their Supabase account.

SUPABASE TABLE	UI EQUIVALENT
teacher_details	Teacher Portal
students	Students Management
payment_history	Fee Management
teacher_payments	Teacher Portal
alumni_tracker	Alumni Section
attendance	attendance
batches	Batch Management
certificates	Certificate Generator
exam	Exam Management
expenses	Expense Tracking
institute_calendar	Academic Calendar
inventory_master	Inventory Manager (Update stock & prices)
inventory_sales	Inventory Manager (Sales record)
leads	Leads management
messages	Message Corner
performance_tracking	Performance Tracking
profiles	Login & Signup
student_allocations	Student Allocation
student_details	Student Management
teachers_attendance	Teachers Attendance
scholarships	Scholarship tracker
ptm_schedule	PTM Schedule

institute_settings

RELATIONS OF TABLES

🏛 MODULE 1: Core System & Authentication

These tables manage who logs in and the global rules of your software.

profiles: This is the User table. When an Owner, Teacher, or Student signs up, their login connects here. It stores their role and binds them to a specific institute_id.

institute_settings: This holds the global settings for a specific institute_id (like the Branch Name and Authorised Signatory).

Connection: The certificates table pulls data from here when a student generates a certificate so the correct owner's name gets printed at the bottom.

🎓 MODULE 2: The Student Ecosystem

This is the heart of your database. The magic thread that connects all these tables together is the roll_number (e.g., IMS-2026-1234) and the internal student_id.

students: The "Master" student table. It holds primary info (Name, Mobile, Course, Total Fees, Pending Fees).

student_details: An "Extension" of the students table. It is linked directly to the students table in a 1-to-1 relationship. It holds extra operational data like course_end_date.

attendance: Logs daily presence.

Connection: Linked to the student via roll_number. If a student is marked absent, the system checks the students table to find the parent's mobile number for WhatsApp alerts.

payment_history: Logs every individual fee payment a student makes.

Connection: Linked to students via roll_number. When a payment is added here, the system automatically subtracts the amount from the pending_fees column in the students table.

exam: Stores individual test scores.

Connection: Linked via roll_number.

performance_tracking: A summary dashboard table.

Connection: It actively calculates data from the exam table (total scores) and attendance table (days present) and displays it on the Student Portal.

certificates: Logs the unique Certificate Number generated for a student.

Connection: Linked via student_id so the certificate number is permanently locked to that specific student.

alumni_tracker: Where students go after they graduate.

Connection: Linked via roll_number. You can only move a student here if their status in the main students table is changed from 'Active' to 'Closed'.

scholarships: Stores scholarship or discount details given to students.

Connection: Linked to students via roll_number. When a scholarship is granted, the system adjusts the total_fees or pending_fees in the students table accordingly. This ensures accurate financial tracking and reporting.

🧑‍🏫 MODULE 3: The Staff Ecosystem

Just like students are tracked by Roll Number, teachers are tracked by their teacher_id (e.g., TCH-1234).

teacher_details: The "Master" staff table. Holds their Name, Qualification, Mobile, and Salary details.

teachers_attendance: Logs daily staff presence.

Connection: Linked to teacher_details via teacher_id. Used by the Admin to see how many days a teacher worked.

teacher_payments: Logs salary payouts (Payslips).

Connection: Linked via teacher_id. When you log a salary here, it generates a payslip using the name and subject from teacher_details.

🔗 MODULE 4: Academics & Linkages (The Bridges)

These tables act as "Bridges" connecting Students and Teachers together.

batches: Defines a classroom.

Connection: Links a teacher_name (from teacher_details) to a specific batch_time and subject_name.

student_allocations: The ultimate bridge table.

Connection: It pairs a teacher_id with a roll_number. This tells the system exactly which student is being taught by which teacher.

institute_calendar: A standalone table that holds Holidays and Events.

Connection: Not directly tied to a student or teacher, but it is displayed on everyone's dashboard via the institute_id.

ptm_schedule: Stores Parent-Teacher Meeting schedules.

Connection 1: Linked to student_allocations to identify which teacher and student are involved in the meeting.

Connection 2: Connected via roll_number (student) and teacher_id (teacher) to ensure the correct participants are notified.

Connection 3: Operates under institute_id so PTM schedules are visible only within the respective institute dashboard.

💰 MODULE 5: Finance & Inventory

These tables manage the money going out and the physical goods being sold.

inventory_master: Your digital catalog. Holds item names (Books, Uniforms) and their current stock levels.

inventory_sales: Logs when a student buys an item.

Connection 1: Linked to students via roll_number to know who bought it.

Connection 2: Linked to inventory_master. When a sale happens here, it automatically tells inventory_master to deduct -1 from the physical stock.

expenses: Your monthly P&L (Profit & Loss) tracker. Logs rent, electricity, and salaries.

Connection: Connected to the inventory_sales table (to show Inventory Income on the dashboard) and operates strictly under the institute_id.

📞 MODULE 6: CRM (Pre-Sales & Comms)

These tables handle things outside the core classroom environment.

leads: A completely standalone table. This holds inquiries (walk-ins, phone calls) from people who are not yet students. Once they pay, they are manually added to the students table.

messages: Your digital Notice Board.

Connection: An Admin writes a message here, tags it with a specific course (e.g., "Mathematics"), and when a student logs into their portal, the system checks the students table to see if their course matches the message tag, displaying it to them if it does.

📋 Summary Cheat Sheet

If you ever get lost, just remember these 3 Golden Rules of your database:

institute_id connects EVERYTHING to the Owner.

roll_number connects STUDENT data (Fees, Attendance, Exams, Sales).

teacher_id connects STAFF data (Salaries, Attendance, Batches).

SEQUENCE IN WHICH TO USE THE 25 TABS

To run your **Class Manager - IMS** project efficiently, you must follow a logical business flow. You cannot track attendance for a student who hasn't been assigned to a teacher, and you cannot assign a student to a teacher if the teacher doesn't exist in the system yet.
Here is the professional 8-phase sequence for using your 25 tabs:

Phase 1: Foundation (Owner Setup)

- **Step 1: [Tab 2] Login System (Signup):** The Owner must sign up first to generate the **Institute ID** (e.g., CM-1234).
- **Step 2: [Tab 11] Teacher Portal:** Add your teachers here. This creates their unique **Teacher ID**, which is required for almost every other module.
- **Step 3: [Tab 5] Batch Management:** Create your batches (e.g., "Morning 9 AM") and link them to a Teacher Name and Subject.

Phase 2: Sales & Admissions

- **Step 4: [Tab 9] Lead Management:** Record inquiries from prospective students. When they pay, you move to the next step.
- **Step 5: [Tab 3] Student Management:** Register the student officially. Generate their **Roll Number**.
- **Step 6: [Tab 23] Scholarship Tracker:** If the student is eligible for a discount, record it here **before** finalizing their fee structure.

Phase 3: Structural Linking (The "Bridge")

- **Step 7: [Tab 18] Student Allocation: CRITICAL STEP.** Use this to link a Student Roll No to a Teacher ID. Without this, your PTM and Performance modules won't know which teacher is responsible for which student.
- **Step 8: [Tab 25] Batch Master:** Run this to see the "Master List." It auto-pulls data from Students, Teachers, and Allocations to confirm your setup is correct.

Phase 4: Daily Operations (Ongoing)

- **Step 9: [Tab 20] Academic Calendar:** Set your holidays and exam dates. This will be visible in the Student Portal.
- **Step 10: [Tab 6] Attendance:** Teachers or Admin mark student presence daily.

- **Step 11: [Tab 17] Teacher Attendance:** Track staff attendance for salary calculations.
- **Step 12: [Tab 22] Inventory Manager:** If you sell books, uniforms, or bags, record the sales here to track stock levels.

Phase 5: Academic Cycle

- **Step 13: [Tab 16] Exam Management:** Enter marks obtained by students in periodic tests.
- **Step 14: [Tab 15] Performance Tracking:** Fetch a student's mobile number to see an automated summary of their attendance % and exam averages.
- **Step 15: [Tab 24] PTM Schedule:** Use the auto-slot generator to book parent meetings and inform them via WhatsApp.

Phase 6: Financial Management

- **Step 16: [Tab 4] Fee Management:** Record fee payments. Use the **UPI QR** generator for instant collections.
- **Step 17: [Tab 8] Fee Reminder System:** Check who is pending and send WhatsApp reminders with one click.
- **Step 18: [Tab 10] Expense Tracking:** Log your rent, electricity, and teacher salaries to see your actual profit.

Phase 7: Student & Staff Experience

- **Step 19: [Tab 12] Students Portal:** Inform students to log in here. They can only use this **after** you have entered their data in Management, Fees, and Exams.
- **Step 20: [Tab 19] Message Corner:** Use this for bulk communication—connecting with the Owner or broadcasting group links.
- **Step 21: [Tab 13] ID Card Generator:** Once a student is "Active," print their ID card.
- **Step 22: [Tab 14] Certificate Generator:** When a student finishes a course, generate their completion certificate.
- **Step 23: [Tab 26] Teacher Reviews:** Students use their Roll Number to provide feedback. This automatically calculates the teacher's average performance score

Phase 8: Analysis & Exit

- **Step 24: [Tab 7] Admin Dashboard:** Use the "Reports Center" to generate high-level charts and printable PDF reports for the month.
- **Step 25: [Tab 21] Alumni Section:** When a student's status changes to "Closed," move their data here to track their college placements or jobs.
- **Step 26: [Tab 1] Home:** Use the Home dashboard for a daily "At-a-glance" view of admissions and collections.

Summary Rule of Thumb:

1. **Add People** (Teachers/Students).
2. **Link People** (Allocation/Batch).
3. **Perform Actions** (Attendance/Exams/Fees).
4. **Review Results** (Dashboard/Portals/Alumni).